

CI/CD for Domo Pro-Code Apps

A Developer Guide

Automating Custom App deployments
with GitHub Actions and ryuu-client

Instance-agnostic guide for the Domo developer community

Table of Contents

1. Overview
2. Repository Structure
3. Creating Domo Designs
4. Configuring GitHub Secrets
5. Production Workflow
6. Preview / Staging Workflow
7. How It All Fits Together
8. Gotchas and Tips
9. Troubleshooting
10. Adding a New App (Checklist)
- A. Appendix: Minimal File Templates
- B. Appendix: Raw Markdown Reference

1. Overview

If you're building Domo Pro-Code (Custom) Apps, you've probably been deploying them by hand -- publishing through the Domo CLI or uploading assets manually. That works fine for a single app, but as your team and app portfolio grow, manual deploys become a bottleneck and a risk. A forgotten step, a wrong design ID, or a publish to production instead of staging can cost hours.

This guide shows you how to set up a CI/CD pipeline using GitHub Actions so that deploying is automatic, repeatable, and safe. By the end, you'll have:

- A **production workflow** that deploys to your prod Domo design every time code merges to `main`
- A **preview/staging workflow** that deploys to a separate staging design on every pull request
- Auto-incrementing version numbers (no more manually bumping `manifest.json`)
- PR comments confirming each staging deploy

What You'll Need

- **GitHub Repository** -- Where your app code lives. Each app gets its own folder.
- **Domo Developer Token** -- Generate one at <https://<instance>.domo.com/admin/developer>. This is how CI authenticates with Domo.
- **Node.js 20+** -- The `ryuu-client` library (Domo's publishing SDK) runs on Node.
- **gh CLI (optional)** -- Handy for setting GitHub secrets from the command line.

Security Note: Your Domo Developer Token is sensitive. Never commit it to your repository. It should only live in GitHub Actions Secrets (or be set temporarily as an environment variable for local one-time scripts).

2. Repository Structure

Each app lives in its own folder. You can put them at the repo root or nest them under a subdirectory -- whatever suits your team. Here's what a minimal app folder looks like:

```
My App/  
+-- manifest.json      # App identity + dataset mappings  
+-- index.html         # Entry point (full HTML document)  
+-- script.js          # Main application logic  
+-- styles.css         # Styling  
+-- domo.js            # Bundled ryuu.js client library  
+-- thumbnail.png      # Required for card creation (200x150 px)  
+-- .domoignore        # Files excluded from publish
```

manifest.json

This is the identity file for your app. It tells Domo what your app is called and which datasets it needs access to.

```
{
  "name": "My App",
  "version": "1.0.0",
  "mapping": [
    { "dataSetId": "aaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee",
      "alias": "sales", "fields": [] }
  ]
}
```

- **name** and **version** are overwritten by CI on every deploy -- you don't need to maintain them by hand.
- **mapping** defines dataset aliases. Your code uses these aliases: `domo.get('/data/v1/sales')`.
- **id** (the design UUID) is **not** stored in the repo. CI injects it from a GitHub secret at deploy time.

domo.js -- The Client Library

Important: The official CDN at `cdn.domo.com/ryuu/domo.js` is unreliable (often returns 500 errors). Always download and bundle a local copy instead.

```
curl -o domo.js https://unpkg.com/ryuu.js@4
```

This gives you the `window.domo` global with `domo.get()`, `domo.post()`, `domo.put()`, `domo.delete()`, and more.

3. Creating Domo Designs

Before CI can deploy your app, you need two Domo "designs" -- think of them as deployment targets. One for production, one for staging. Each design gets a unique UUID that you'll store as a GitHub secret.

There are two ways to create designs: a bootstrap script (recommended) or manual API calls.

Option A: Bootstrap Script (Recommended)

Create a file called `create-designs.js` in your app folder. This is a one-time script that creates both designs, does the initial publish, and saves the design IDs as GitHub secrets. See the full script in the Markdown Appendix at the end of this guide.

Run it like this:

```
npm install --no-save ryuu-client@latest  
DOMO_ACCESS_TOKEN=<your-token> node create-designs.js
```

The script will:

1. Create PROD and STG designs via the Domo API
2. Do an initial publish and release for each (so the designs are usable)
3. Store the design UUIDs as GitHub secrets using the gh CLI

Tip: Add `create-designs.js` to your `.domoignore` file so it doesn't get published to Domo.

Option B: Manual API Calls

If you prefer curl, you can create designs directly:

```
curl -X POST "https://<instance>.domo.com/api/apps/v1/designs" \  
-H "X-Domo-Developer-Token: <your-token>" \  
-H "Content-Type: application/json" \  
-d '{"name": "PROD - My App", "version": "1.0.0"}'
```

Note the `id` from the JSON response. Repeat for the STG design with a different name. Then store both UUIDs as GitHub secrets (next section).

Important: The `version` field is required in the POST body. Omitting it returns a 400 error. Also, designs are empty after creation -- you must do an initial publish + release before you can create cards from them in Domo.

4. Configuring GitHub Secrets

GitHub Actions needs three pieces of information to deploy your app, all stored as repository secrets. Secrets are encrypted and never exposed in logs or to forks.

Secret Name	What It Is
DOMO_ACCESS_TOKEN	Your Domo Developer Token
PROD_MYAPP	Production design UUID (from step 3)
STG_MYAPP	Staging design UUID (from step 3)

Setting secrets via the CLI:

```
gh secret set DOMO_ACCESS_TOKEN -R <org>/<repo>
gh secret set PROD_MYAPP -R <org>/<repo>
gh secret set STG_MYAPP -R <org>/<repo>
```

Or use the GitHub web UI: **Settings > Secrets and variables > Actions > New repository secret**.

Watch out: `gh secret set` auto-uppercases secret names. `PROD_MyApp` becomes `PROD_MYAPP`. To be safe, reference both casings in your workflow:

```
DESIGN_ID: ${ secrets.PROD_MyApp || secrets.PROD_MYAPP }
```

5. Production Workflow

The production workflow runs whenever code is pushed (merged) to the main branch and changes fall under your app's folder. It publishes a new version to your PROD design, making it immediately live on any Domo cards using that design.

Create a file at `.github/workflows/myapp-prod.yml`. The full YAML is in the Markdown Appendix, but here's what it does:

1. **Checks out** your code and sets up Node.js 20
2. **Installs ryuu-client** (Domo's publishing SDK)
3. **Copies your app** to a temp directory (to avoid path issues with spaces)
4. **Injects the design ID** from GitHub Secrets into `manifest.json`
5. **Computes the next version number** by reading existing versions and incrementing the major
6. **Uploads all assets** via `ryuu-client`
7. **Releases the version** via a raw HTTPS PUT call (more reliable than `ryuu-client`'s built-in release)

Key environment variables in the workflow:

Variable	Purpose
DOMO_ACCESS_TOKEN	From secrets -- authenticates with Domo API
DESIGN_ID	From secrets -- which design to publish to
INSTANCE	Your Domo subdomain (e.g., mycompany)
APP_DIR	Path to your app folder in the repo
MANIFEST_NAME	Display name (prefixed PROD or STG)

6. Preview / Staging Workflow

The preview workflow is nearly identical to production, but it triggers on pull requests instead of pushes to main. This gives your team a safe way to test changes on a staging card before merging.

Create a file at `.github/workflows/myapp-preview.yml`. It does everything the production workflow does, plus:

- Uses the **STG** design ID instead of PROD
- Writes a deployed output so the next step knows whether it succeeded
- Posts an **automated comment on the PR** with the deployed version number

Tip: Create a staging dashboard in Domo with a card built from your STG design. Map the same datasets. When the preview workflow deploys, just refresh the card to see your changes. This is a great way to do QA before merging.

7. How It All Fits Together

Here's the full developer workflow from code change to live Domo card:

Step	What Happens	Where
1	Developer pushes to a feature branch	GitHub
2	Developer opens a Pull Request	GitHub
3	Preview workflow fires automatically	GitHub Actions
4	App is published to the STG design	Domo (staging)
5	PR gets an automated comment with deploy status	GitHub PR
6	Team reviews code + tests on staging card	Domo + GitHub
7	PR is merged to main	GitHub
8	Production workflow fires automatically	GitHub Actions
9	App is published to the PROD design -- live!	Domo (production)

8. Gotchas and Tips

These are hard-won lessons from deploying many Domo apps. Read through these before you hit them the hard way.

Folder Names with Spaces

ryuu-client's `uploadAllAssets()` breaks on paths containing spaces or special characters (&, (, etc.). The workflow templates handle this by copying to a clean temp directory before publishing. If your folder names are simple alphanumeric, you can skip the copy step.

Version Format Surprises

`getVersions()` returns an array of **strings** like `["0.0.1", "1", "2.0.0"]`, not objects. Always parse carefully: `parseInt(String(v).split('.')[0], 10)`

Release Step is Mandatory

Uploading assets is not enough to make a version live. You must also call the release endpoint. We use a raw HTTPS PUT instead of `ryuu-client.release()` because the latter can crash on the empty 200 response body.

The CDN is Broken

`cdn.domo.com/ryuu/domo.js` often returns 500 errors. Always download from unpkg and commit a local `domo.js` to your repo.

Token Management

Generate tokens at `https://<instance>.domo.com/admin/developer`. Store as `DOMO_ACCESS_TOKEN` in GitHub Secrets. Tokens are **not** available in your local shell by default -- set them temporarily as an env var when running one-time scripts.

9. Troubleshooting

Quick reference for common issues and their fixes.

Symptom	Cause	Fix
uploadAllAssets fails with path error	Spaces or special chars in folder path	Copy to temp directory before publish
getVersions returns unexpected values	Version strings aren't always semver	Use the <code>parseInt(split('.')[0])</code> pattern
Design exists but can't create cards	No version released yet	Run initial publish + release
400 on design creation	Missing version field in POST body	Always include "version": "1.0.0"
Secret not found in workflow	gh secret set uppercased the name	Use fallback: <code>secrets.X secrets.X_UPPER</code>
Release returns non-200	Token expired or wrong design ID	Regenerate token; verify design UUID
Empty 200 on release crashes ryuu-client	Known ryuu-client bug	Use raw HTTPS for release instead

10. Adding a New App (Checklist)

Every time you onboard a new Domo Pro-Code app, follow these steps:

1. Create the app folder with `manifest.json`, `index.html`, `script.js`, `styles.css`, `domo.js`, and `thumbnail.png`
2. Create a `.domoignore` file to exclude dev-only files from publish
3. Run `create-designs.js` (or use `curl`) to create PROD and STG designs
4. Store the design UUIDs as `PROD_<APPNAME>` and `STG_<APPNAME>` GitHub secrets
5. Create the two workflow files (`.github/workflows/<app>-prod.yml` and `<app>-preview.yml`)
6. Push to main to trigger the first production deploy
7. In Domo, create cards from the PROD and STG designs and map your datasets

Appendix A: Minimal File Templates

.domoignore

```
create-designs.js
*.test.js
README.md
.domoignore
docs/
```

manifest.json

```
{
  "name": "My App",
  "version": "1.0.0",
  "mapping": [
    { "dataSetId": "your-dataset-uuid",
      "alias": "mydata", "fields": [] }
  ]
}
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>My App</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div id="app"></div>
  <script src="domo.js"></script>
  <script src="script.js"></script>
</body>
</html>
```

Appendix B: Raw Markdown Reference

The complete source markdown is included below for easy copy-paste of code blocks, workflow YAML, and configuration snippets.

```
# CI/CD for Domo Pro-Code Apps with GitHub Actions

A developer guide for automating Domo Custom App deployments using GitHub Actions and `ryuu-client`.

---

## Overview

This guide walks through setting up automated CI/CD for Domo Pro-Code (Custom) Apps. The approach uses GitHub Actions to publish app assets to Domo designs on every

**What you'll end up with:**

- A **production workflow** that deploys to your prod Domo design on merge to `main`
- A **preview/staging workflow** that deploys to a separate staging design on every PR
- Auto-incrementing version numbers
- PR comments with deployment status

**Prerequisites:**

- A GitHub repository for your Domo apps
- A Domo Developer Token – generate one at `https://&lt;instance>.domo.com/admin/developer`
- Node.js 20+
- The `gh` CLI (optional, for automating secret setup)

---

## 1. Repository Structure

Each app lives in its own folder at the repo root (or under a subdirectory like `domo/`). A minimal app folder looks like:

```
My App/
 manifest.json # App identity + dataset mappings
 index.html # Entry point
 script.js # App logic
 styles.css # Styling
 domo.js # Bundled ryuu.js client library
 thumbnail.png # Required for card creation (200x150)
 .domoignore # Files excluded from publish
```

### manifest.json

```json
{
 "name": "My App",
 "version": "1.0.0",
 "mapping": [
 { "dataSetId": "aaaaaaa-bbbb-cccc-dddd-eeeeeeeeeee", "alias": "sales", "fields": [] }
]
}
```

Key points:
- `name` and `version` are managed by CI – they get overwritten on each deploy.
```

- `mapping` defines dataset aliases your code uses: `domo.get('/data/v1/sales')`.
- `id` (the design UUID) is **not** stored in the repo. CI injects it from a GitHub secret.

domo.js

The CDN at `cdn.domo.com/ryuu/domo.js` is unreliable. Always download and bundle locally:

```
```bash
curl -o domo.js https://unpkg.com/ryuu.js@4
```
```

This gives you the `window.domo` global with `domo.get()`, `domo.post()`, `domo.put()`, `domo.delete()`, etc.

2. Create Domo Designs

Every app needs two Domo designs – one for production, one for staging. Designs are created via the Domo API.

Option A: Script (Recommended)

Create a `create-designs.js` in your app folder. This is a one-time bootstrap script:

```
```javascript
#!/usr/bin/env node
const https = require('https');
const fs = require('fs');
const path = require('path');
const os = require('os');

// --- Config ---
const INSTANCE = '<your-instance>'; // e.g., 'mycompany'
const APP_NAME = 'My App';
const TOKEN = process.env.DOMO_ACCESS_TOKEN;
const SECRET_PREFIX = 'MYAPP'; // GitHub secret prefix (no spaces)
const GITHUB_REPO = '<org>/<repo>'; // e.g., 'myorg/domo-apps'

if (!TOKEN) {
 console.error('Set DOMO_ACCESS_TOKEN env var first.');
```

```
 process.exit(1);
}

// --- Helpers ---
function createDesign(name) {
 return new Promise((resolve, reject) => {
 const body = JSON.stringify({ name, version: '1.0.0' });
 const req = https.request({
 hostname: `${INSTANCE}.domo.com`,
 path: '/api/apps/v1/designs',
 method: 'POST',
 headers: {
 'X-Domo-Developer-Token': TOKEN,
 'Content-Type': 'application/json',
 'Content-Length': body.length
 }
 })
 })
}
```

```

 }, res => {
 let data = '';
 res.on('data', c => data += c);
 res.on('end', () => {
 if (res.statusCode >= 300) return reject(new Error(`${res.statusCode}: ${data}`));
 resolve(JSON.parse(data));
 });
 });
 });
 req.on('error', reject);
 req.write(body);
 req.end();
});
}

function releaseVersion(designId, version) {
 return new Promise((resolve, reject) => {
 const req = https.request({
 hostname: `${INSTANCE}.domo.com`,
 path: `/api/apps/v1/designs/${designId}/versions/${version}/release`,
 method: 'PUT',
 headers: {
 'X-Domo-Developer-Token': TOKEN,
 'Content-Length': 0
 }
 });
 }, res => {
 res.on('data', () => {});
 res.on('end', () => {
 if (res.statusCode >= 300) return reject(new Error(`Release failed: ${res.statusCode}`));
 resolve();
 });
 });
 });
 req.on('error', reject);
 req.end();
});
}

async function main() {
 // Create both designs
 const prod = await createDesign(`PROD - ${APP_NAME}`);
 console.log(`PROD design: ${prod.id}`);

 const stg = await createDesign(`STG - ${APP_NAME}`);
 console.log(`STG design: ${stg.id}`);

 // Copy to temp dir (avoid spaces in path breaking ryuu-client)
 const slug = APP_NAME.replace(/^[a-zA-Z0-9]/g, '');
 const tmpDir = path.join(os.tmpdir(), slug);
 const srcDir = __dirname;

 if (fs.existsSync(tmpDir)) fs.rmSync(tmpDir, { recursive: true });
 fs.cpSync(srcDir, tmpDir, { recursive: true });

 // Initial publish for each design
 process.env.DOMO_ACCESS_TOKEN = TOKEN;
 const domo = require('ryuu-client');

```

```

for (const [label, designId] of [['PROD', prod.id], ['STG', stg.id]]) {
 const manifest = JSON.parse(fs.readFileSync(path.join(tmpDir, 'manifest.json'), 'utf8'));
 manifest.id = designId;
 manifest.name = `${label} - ${APP_NAME}`;
 manifest.version = '1.0.0';
 manifest.ignore = ['create-designs.js', '*.test.js', 'README.md', '.domoignore'];
 fs.writeFileSync(path.join(tmpDir, 'manifest.json'), JSON.stringify(manifest, null, 2));

 const cwd = process.cwd();
 process.chdir(tmpDir);
 try {
 await domo.uploadAllAssets(manifest);
 await releaseVersion(designId, '1.0.0');
 console.log(`${label}: published v1.0.0`);
 } catch (err) {
 console.error(`${label} publish failed:`, err.message);
 }
 process.chdir(cwd);
}

// Store as GitHub secrets (requires `gh` CLI)
const { execSync } = require('child_process');
for (const [label, id] of [['PROD', prod.id], ['STG', stg.id]]) {
 const secretName = `${label}_${SECRET_PREFIX}`;
 try {
 execSync(`echo "${id}" | gh secret set ${secretName} -R ${GITHUB_REPO}`, { stdio: 'pipe' });
 console.log(`Secret ${secretName} saved.`);
 } catch {
 console.log(`Could not set secret ${secretName} automatically.`);
 console.log(`Run manually: gh secret set ${secretName} -R ${GITHUB_REPO}`);
 console.log(`Value: ${id}`);
 }
}
}

main().catch(err => { console.error(err); process.exit(1); });
...

Run it:

```bash
npm install --no-save ryuu-client@latest
DOMO_ACCESS_TOKEN=DDCI... node create-designs.js
```

Add `create-designs.js` to your `.domoignore` so it doesn't get published.

Option B: Manual API Calls

```bash
curl -X POST "https://<instance>.domo.com/api/apps/v1/designs" \
  -H "X-Domo-Developer-Token: " \
  -H "Content-Type: application/json" \
  -d '{"name": "PROD - My App", "version": "1.0.0"}'

```

```

---

Note the `id` from the response. Repeat for the STG design. Then store both as GitHub secrets (see step 3).

---

## 3. Configure GitHub Secrets

You need three secrets in your repository:

| Secret | Description |
|-----|-----|
| `DOMO_ACCESS_TOKEN` | Your Domo Developer Token |
| `PROD_MYAPP` | Production design UUID |
| `STG_MYAPP` | Staging design UUID |

Set them via the GitHub UI (Settings > Secrets and variables > Actions) or the CLI:

```bash
gh secret set DOMO_ACCESS_TOKEN -R <org>/<repo>;
gh secret set PROD_MYAPP -R <org>/<repo>;
gh secret set STG_MYAPP -R <org>/<repo>;
```

**Heads up:** `gh secret set` auto-uppercases secret names. `PROD_MyApp` becomes `PROD_MYAPP`. Reference both casings in your workflow as a safety net:

```yaml
DESIGN_ID: ${ secrets.PROD_MyApp || secrets.PROD_MYAPP }
```

---

## 4. Production Workflow

Create `.github/workflows/myapp-prod.yml`:

```yaml
name: "My App - Production Deploy"

on:
 push:
 branches: [main]
 paths:
 - "My App/**"

jobs:
 deploy:
 runs-on: ubuntu-latest
 env:
 DOMO_ACCESS_TOKEN: ${ secrets.DOMO_ACCESS_TOKEN }
 DESIGN_ID: ${ secrets.PROD_MYAPP || secrets.PROD_MyApp }
 INSTANCE: "<your-instance>"
 APP_DIR: "My App"
 MANIFEST_NAME: "PROD - My App"
 steps:

```



```

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
 with:
 node-version: "20"

- name: Install ryuu-client
 run: npm install --no-save ryuu-client@latest

- name: Publish to Domo
 env:
 NODE_PATH: ${ github.workspace }/node_modules
 run: |
 node -e "
 const fs = require('fs');
 const path = require('path');
 const https = require('https');
 const os = require('os');

 const designId = process.env.DESIGN_ID;
 if (!designId) { console.log('No design ID - skipping. '); process.exit(0); }

 // Copy to temp dir (spaces in folder names break ryuu-client)
 const slug = process.env.APP_DIR.replace(/^[^a-zA-Z0-9]/g, '');
 const srcDir = path.resolve(process.env.APP_DIR);
 const tmpDir = path.join(os.tmpdir(), slug + '-' + Date.now());
 fs.cpSync(srcDir, tmpDir, { recursive: true });

 const domo = require('ryuu-client');

 // Read and update manifest
 const manifestPath = path.join(tmpDir, 'manifest.json');
 const manifest = JSON.parse(fs.readFileSync(manifestPath, 'utf8'));
 manifest.id = designId;
 manifest.name = process.env.MANIFEST_NAME;

 // Compute next major version
 async function getNextVersion() {
 const versions = await domo.getVersions(manifest.id);
 const majors = versions
 .map(v => parseInt(String(v).split('.')[0], 10))
 .filter(n => Number.isFinite(n));
 const max = majors.length ? Math.max(...majors) : 0;
 return (max + 1) + '.0.0';
 }

 // Release via raw HTTPS (ryuu-client release() can choke on empty body)
 function releaseVersion(version) {
 return new Promise((resolve, reject) => {
 const req = https.request({
 hostname: process.env.INSTANCE + '.domo.com',
 path: '/api/apps/v1/designs/' + designId + '/versions/' + version + '/release',
 method: 'PUT',
 headers: {
 'X-Domo-Developer-Token': process.env.DOMO_ACCESS_TOKEN,
 'Content-Length': 0
 }
 });
 });
 }

```

```

 }
 }, res => {
 res.on('data', () => {});
 res.on('end', () => res.statusCode < 300 ? resolve() : reject(new Error('Release ' + res.statusCode)));
 });
 req.on('error', reject);
 req.end();
 });
}

(async () => {
 const version = await getNextVersion();
 manifest.version = version;
 manifest.ignore = ['create-designs.js', '*.test.js', 'README.md', '.demoignore'];
 fs.writeFileSync(manifestPath, JSON.stringify(manifest, null, 2));

 process.chdir(tmpDir);
 await domo.uploadAllAssets(manifest);
 await releaseVersion(version);
 console.log('Deployed v' + version + ' to PROD');
})();
}).catch(err => { console.error(err); process.exit(1); });
"
...

5. Preview/Staging Workflow

Create `.github/workflows/myapp-preview.yml`:

```yaml
name: "My App - PR Preview"

on:
  pull_request:
    types: [opened, synchronize, reopened]
    paths:
      - "My App/**"

jobs:
  preview:
    runs-on: ubuntu-latest
    env:
      DOMO_ACCESS_TOKEN: ${ secrets.DOMO_ACCESS_TOKEN }
      DESIGN_ID: ${ secrets.STG_MYAPP || secrets.STG_MyApp }
      INSTANCE: "<your-instance>"
      APP_DIR: "My App"
      MANIFEST_NAME: "STG - My App"
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"

      - name: Install ryuu-client

```

```

run: npm install --no-save ryyu-client@latest

- name: Publish to Domo
  id: deploy
  env:
    NODE_PATH: ${ github.workspace }}/node_modules
  run: |
    node -e "
    const fs = require('fs');
    const path = require('path');
    const https = require('https');
    const os = require('os');

    const designId = process.env.DESIGN_ID;
    if (!designId) {
      console.log('No staging design ID - skipping.');
```

```

      fs.appendFileSync(process.env.GITHUB_OUTPUT, 'deployed=false\n');
      process.exit(0);
    }

    const slug = process.env.APP_DIR.replace(/^[a-zA-Z0-9]/g, '');
    const srcDir = path.resolve(process.env.APP_DIR);
    const tmpDir = path.join(os.tmpdir(), slug + '-' + Date.now());
    fs.cpSync(srcDir, tmpDir, { recursive: true });

    const domo = require('ryyu-client');
    const manifestPath = path.join(tmpDir, 'manifest.json');
    const manifest = JSON.parse(fs.readFileSync(manifestPath, 'utf8'));
    manifest.id = designId;
    manifest.name = process.env.MANIFEST_NAME;

    async function getNextVersion() {
      const versions = await domo.getVersions(manifest.id);
      const majors = versions
        .map(v => parseInt(String(v).split('.')[0], 10))
        .filter(n => Number.isFinite(n));
      const max = majors.length ? Math.max(...majors) : 0;
      return (max + 1) + '.0.0';
    }

    function releaseVersion(version) {
      return new Promise((resolve, reject) => {
        const req = https.request({
          hostname: process.env.INSTANCE + '.domo.com',
          path: '/api/apps/v1/designs/' + designId + '/versions/' + version + '/release',
          method: 'PUT',
          headers: {
            'X-Domo-Developer-Token': process.env.DOMO_ACCESS_TOKEN,
            'Content-Length': 0
          }
        }, res => {
          res.on('data', () => {});
          res.on('end', () => { res.statusCode < 300 ? resolve() : reject(new Error('Release ' + res.statusCode)); });
        });
        req.on('error', reject);
      });
    }

```

```

        req.end();
    });
}

(async () => {
    const version = await getNextVersion();
    manifest.version = version;
    manifest.ignore = ['create-designs.js', '*.test.js', 'README.md', '.domoignore'];
    fs.writeFileSync(manifestPath, JSON.stringify(manifest, null, 2));

    process.chdir(tmpDir);
    await domo.uploadAllAssets(manifest);
    await releaseVersion(version);
    console.log('Deployed v' + version + ' to STG');
    fs.appendFileSync(process.env.GITHUB_OUTPUT, 'deployed=true\nversion=' + version + '\n');
})();catch(err => { console.error(err); process.exit(1); });
"

- name: Comment on PR
  if: steps.deploy.outputs.deployed == 'true'
  uses: actions/github-script@v7
  with:
    script: |
      github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: `**Staging Deploy** - v${process.env.VERSION} published to STG design.\n\nView it on a staging card in your Domo instance.`
      })
  env:
    VERSION: ${ steps.deploy.outputs.version }
...

---

## 6. How It All Fits Together

...
Developer pushes to feature branch
  ▼
Opens Pull Request
  ▼
Preview workflow fires ■■■ Publishes to STG design
  ▼
  ■ PR gets a comment
  ■ with deploy status
  ▼
Code review + QA on staging card
  ▼
Merge to main
  ■

```

```

▼
Prod workflow fires ■■■ Publishes to PROD design
      ■
      ▼
Live in Domo
...

### Viewing Staged Changes

1. In Domo, create a card from your **STG** design on a staging dashboard.
2. Map the same datasets to the card as you would for production.
3. When the preview workflow deploys, refresh the card to see changes.

### Viewing Production

Cards created from the **PROD** design auto-update when a new version is released.

---

## 7. Gotchas and Tips

### Folder names with spaces

`ryuu-client`'s `uploadAllAssets()` breaks on paths with spaces or special characters (`&`, `(`, etc.). The workflows above handle this by copying to a temp dire

### Version format

`getVersions()` returns an array of **strings** like `["0.0.1", "1", "2.0.0"]`, not objects. Always parse carefully:

```javascript
parseInt(String(v).split('.')[0], 10)
```

### Release step is required

Uploading assets alone doesn't make a version live. You must explicitly call the release endpoint. The raw HTTPS approach is more reliable than `ryuu-client.release`

### CDN is broken

`cdn.domo.com/ryuu/domo.js` returns 500 errors. Always download from unpkg and commit `domo.js` to your repo:

```bash
curl -o domo.js https://unpkg.com/ryuu.js@4
```

### Token management

- Generate tokens at `https://&lt;instance&gt;.domo.com/admin/developer`
- Store as `DOMO_ACCESS_TOKEN` in GitHub Actions secrets
- Tokens are **not** available in your local shell by default – set them explicitly when running scripts locally

### Adding a new app

For each new app:

```

1. Create the app folder with `manifest.json`, `index.html`, etc.
2. Run `create-designs.js` (or use curl) to create PROD and STG designs
3. Store design UUIDs as `PROD\_<APPNAME>` and `STG\_<APPNAME>` secrets
4. Create the two workflow files with the appropriate path filters
5. Push to `main` to trigger the first production deploy
6. Create cards from the designs in Domo and map datasets

```
### .domoignore
```

Keeps dev-only files out of the published design:

```
...
create-designs.js
*.test.js
README.md
.domoignore
docs/
...
```

The workflows also set `manifest.ignore` programmatically as a second layer.

```
---
```

8. Troubleshooting

| Symptom | Cause | Fix |
|--|--|--|
| `uploadAllAssets` fails with path error | Spaces or special chars in folder path | Copy to temp directory before publish |
| `getVersions` returns unexpected values | Version strings aren't always semver | Use the `parseInt(split('.')[0])` pattern |
| Design exists but can't create cards | No version released yet | Run initial publish + release |
| 400 on design creation | Missing `version` field in POST body | Always include `"version": "1.0.0"` |
| Secret not found in workflow | `gh secret set` uppercased the name | Use fallback: `secrets.PROD_Foo` or `secrets.PROD_F00` |
| Release returns non-200 | Token expired or wrong design ID | Regenerate token; verify design UUID |
| Empty 200 on release crashes ryuu-client | Known ryuu-client bug | Use raw HTTPS for release instead |

```
---
```

Appendix: Minimal File Templates

```
### .domoignore
```

```
...
create-designs.js
*.test.js
README.md
.domoignore
...
```

```
### manifest.json
```

```
```json
{
 "name": "My App",
 "version": "1.0.0",
 "mapping": [
```

```
 { "dataSetId": "your-dataset-uuid", "alias": "mydata", "fields": [] }
]
}
```

```
index.html
```

```
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My App</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div id="app">
    <script src="domo.js">
    <script src="script.js">
  </div>
</body>
</html>
```
```

